

AIM: Data Wrangling with pandas: Manipulate and transform data to suit the analysis needs.

Theory

1. What is Data Wrangling? Explain the difference between Data Wrangling and Data Preprocessing.

Ans: Data wrangling refers to the process of cleaning, transforming, and arranging raw data into a well-structured format so it can be easily analyzed. In real-life scenarios, data is often messy, incomplete, or contains errors, so data wrangling helps improve its quality by fixing missing values, removing duplicates, and converting data into a proper format. Data wrangling and data preprocessing differ in their purpose and when they are used. Data wrangling is mainly focused on preparing raw data for analysis by organizing and cleaning it. On the other hand, data preprocessing is used to prepare data specifically for machine learning models. It involves techniques like scaling, normalization, and encoding. In short, data wrangling makes the data ready for analysis, while data preprocessing prepares it for building and training models.

2. What is the importance of Exploratory Data Analysis (EDA) in Pandas?

Ans: Exploratory Data Analysis (EDA) is a key step in data analysis that helps us understand the dataset and its main features. It is used to find missing values, recognize patterns and relationships, and identify any unusual values or errors. By performing EDA, we can better understand the data before moving on to advanced analysis or model building. In Pandas, functions like `head()`, `info()`, and `describe()` are widely used for EDA. These functions help in viewing the dataset, checking data types, and getting a summary of statistical details. Overall, EDA helps in making better decisions and ensures that the data is properly prepared for further analysis.

3. What are the main differences between Series, DataFrame, and Panel?

Ans: In Pandas, Series, DataFrame, and Panel are different data structures used to store and work with data. A Series is a one-dimensional structure that holds data of any type and is similar to a single column. A DataFrame is a two-dimensional structure with rows and columns, which makes it ideal for handling tabular data and performing analysis. A Panel is a three-dimensional data structure that was used for working with multi-dimensional data. However, it is now deprecated and not commonly used in recent versions of Pandas. In short, Series is used for simple one-dimensional data, DataFrame is the most commonly used structure for analysis, and Panel was used for higher-dimensional data.

4. Explain the use of `isnull()`, `notnull()`, `dropna()`, and `fillna()`.

Ans: In Pandas, there are different functions available to manage missing data. The `isnull()` function helps identify missing values by returning True for those entries. In contrast, `notnull()` returns True for values that are present. The `dropna()` function is used to remove rows or columns that contain missing values, especially when those missing entries are not very important. On the other hand, `fillna()` is used to replace missing values with a specific value, such as an average, median, or a fixed number. These functions are very important for data cleaning and help ensure that the analysis results are accurate and reliable.

5. What is column transformation and why is it important in data analysis?

Ans: Column transformation refers to modifying existing columns or creating new ones in a dataset to make the data more useful for analysis. This can include tasks like changing data types, generating new columns from existing data, performing calculations, and converting categorical values into numerical form. It is important because it improves the quality and consistency of data, making it easier to analyze. It also helps in preparing the dataset for machine learning models and makes results easier to understand and visualize. Overall, column transformation is an essential step in effective data analysis.

```
import pandas as pd
import numpy as np
# Create Dataset (Raw data with missing values)
data = {
    'Name': ['Vaishnavi', 'Ruchi', 'Bhumika', 'Mansi', 'Trupti'],
    'Age': [20, 22, np.nan, 19, 23],
    'Marks': [86, np.nan, 79, 82, 82]
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
#Exploratory Data Analysis (EDA)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
#Check missing value
print("\nMissing Values (True = Missing):")
print(df.isnull())
print("\nCount of Missing Values:")
print(df.isnull().sum())
#Use of notnull()
print("\nNon-Missing Values (True = Present):")
print(df.notnull())
```

```
#Remove Missing Values using dropna()
df_drop = df.dropna()
print("\nDataset after removing missing values:")
print(df_drop)
# Fill Age with mean
df['Age'] = df['Age'].fillna(df['Age'].mean())
# Fill Marks with mean
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
print("\nDataset after filling missing values:")
print(df)
#Column Transformation
# Create new column
df['Total'] = df['Marks'] + 10
# Convert datatype of Age to integer
df['Age'] = df['Age'].astype(int)
print("\nDataset after column transformation:")
print(df)
```

```
dtypes: float64(2), object(1)
memory usage: 252.0+ bytes
None
```

Statistical Summary:

	Age	Marks
count	4.000000	4.000000
mean	21.000000	82.250000
std	1.825742	2.872281
min	19.000000	79.000000
25%	19.750000	81.250000
50%	21.000000	82.000000
75%	22.250000	83.000000
max	23.000000	86.000000

Missing Values (True = Missing):

	Name	Age	Marks
0	False	False	False
1	False	False	True
2	False	True	False
3	False	False	False
4	False	False	False

Count of Missing Values:

Name	0
Age	1
Marks	1

dtype: int64

Non-Missing Values (True = Present):

	Name	Age	Marks
0	True	True	True
1	True	True	False
2	True	False	True
3	True	True	True
4	True	True	True

Dataset after removing missing values:

	Name	Age	Marks
0	Vaishnavi	20.0	86.0
3	Mansi	19.0	82.0
4	Trupti	23.0	82.0

Dataset after filling missing values:

	Name	Age	Marks
0	Vaishnavi	20.0	86.00
1	Ruchi	22.0	82.25
2	Bhumika	21.0	79.00
3	Mansi	19.0	82.00
4	Trupti	23.0	82.00

Dataset after column transformation:

	Name	Age	Marks	Total
0	Vaishnavi	20	86.00	96.00
1	Ruchi	22	82.25	92.25
2	Bhumika	21	79.00	89.00
3	Mansi	19	82.00	92.00
4	Trupti	23	82.00	92.00

Data Wrangling Using IRIS Dataset

```
import pandas as pd
import numpy as np
```

```
# Load dataset from URL
url = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"
df = pd.read_csv(url)
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# First 5 rows
df.head()
# Last 5 rows
df.tail()
# Dataset information
df.info()
# Statistical summary
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sepal_length    150 non-null    float64
1   sepal_width     150 non-null    float64
2   petal_length    150 non-null    float64
3   petal_width     150 non-null    float64
4   species         150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.isnull().sum()
```

```
0
sepal_length  0
sepal_width   0
petal_length  0
petal_width   0
species       0
dtype: int64
```

```
# Introduce missing values manually
df.loc[0, 'sepal_length'] = np.nan
df.loc[1, 'petal_width'] = np.nan
df.isnull().sum()
```

```

0
sepal_length 1
sepal_width 0
petal_length 0
petal_width 1
species 0

```

dtype: int64

```

# Fill missing values with mean
df['sepal_length'].fillna(df['sepal_length'].mean())
df['petal_width'].fillna(df['petal_width'].mean())
df.isnull().sum()

```

```

0
sepal_length 1
sepal_width 0
petal_length 0
petal_width 1
species 0

```

dtype: int64

```
df.notnull().head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	False	True	True	True	True
1	True	True	True	False	True
2	True	True	True	True	True
3	True	True	True	True	True
4	True	True	True	True	True

```

# Create new column
df['sepal_area'] = df['sepal_length'] * df['sepal_width']
df.head()

```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	NaN	3.5	1.4	0.2	setosa	NaN
1	4.9	3.0	1.4	NaN	setosa	14.70
2	4.7	3.2	1.3	0.2	setosa	15.04
3	4.6	3.1	1.5	0.2	setosa	14.26
4	5.0	3.6	1.4	0.2	setosa	18.00

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

# Filter rows where sepal_length > 5
df[df['sepal_length'] > 5]

```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
5	5.4	3.9	1.7	0.4	setosa	21.06
10	5.4	3.7	1.5	0.2	setosa	19.98
14	5.8	4.0	1.2	0.2	setosa	23.20

```
# Average values by species
df.groupby('species').mean()
```

...	sepal_length	sepal_width	petal_length	petal_width	sepal_area	..
species	6.7	3.0	5.2	2.3	virginica	20.10
setosa	5.004082	3.418	1.464	0.244898	17.195714	.75
versicolor	5.936000	3.0	2.770	5.2	4.260	21.026000
virginica	6.588000	2.974	5.552	2.026000	19.684600	.08
149	5.9	3.0	5.1	1.8	virginica	17.70

```
# Sort by sepal_length
df.sort_values(by='sepal_length', ascending=False).head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
131	7.9	3.8	6.4	2.0	virginica	30.02
118	7.7	2.6	6.9	2.3	virginica	20.02
117	7.7	3.8	6.7	2.2	virginica	29.26
122	7.7	2.8	6.7	2.0	virginica	21.56
135	7.7	3.0	6.1	2.3	virginica	23.10

```
print("Final Dataset:")
df.head()
```

Final Dataset:

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	NaN	3.5	1.4	0.2	setosa	NaN
1	4.9	3.0	1.4	NaN	setosa	14.70
2	4.7	3.2	1.3	0.2	setosa	15.04
3	4.6	3.1	1.5	0.2	setosa	14.26
4	5.0	3.6	1.4	0.2	setosa	18.00

Next steps: [Generate code with df](#) [New interactive sheet](#)

Conclusion:

In this practical exercise, the Iris dataset was processed using Pandas for data wrangling. Exploratory Data Analysis (EDA) was conducted to examine the dataset and gain insights into its structure and features. Missing values were identified and handled with functions like `isnull()` and `fillna()`. Additionally, column transformation techniques were applied to generate new features and improve data quality. Consequently, the dataset was cleaned, well-organized, and prepared for further analytical tasks.